

PHẠM HUY HOÀNG

CODE DẠO KỸ SỰ

Lập trình viên đâu phải chỉ biết code

PHẠM HUY HOÀNG

CODE DẠO KỸ SỰ



Trithuctrebooks

CÔNG TY TNHH SÁCH VÀ TRUYỀN THÔNG VIỆT NAM
ĐC: Số nhà 23/56/376 đường Bưởi - P.Vĩnh Phúc - Q.Ba Đình - Hà Nội
ĐT: 04. 6293 2066 - Fax: 04. 3838 9613
E-mail: trithuctrebooks@gmail.com

CODE DẠO KỸ SỰ

Giá: 159.000đ

ISBN: 978-604-943-417-4



trithuctrebooks

PHẠM HUY HOÀNG

**CODE DẠO KÍ SỰ
LẬP TRÌNH VIÊN ĐÂU PHẢI
CHỈ BIẾT CODE**

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

NHÀ XUẤT BẢN TRI THỨC

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

LỜI GIỚI THIỆU

Chào các bạn, mình là Phạm Huy Hoàng, chủ một blog IT mang tên Tôi đi code dạo.

Hiện nay, ngành IT nói chung và lập trình nói riêng đang trở thành một ngành hot, được khá nhiều bạn sinh viên lựa chọn. Tuy nhiên, so với nước ngoài, các bạn sinh viên Việt Nam chịu khá nhiều thiệt thòi vì thiếu những tấm gương và tài liệu để học hỏi. Thuở còn là sinh viên, mình cũng từng có những thắc mắc, trăn trở về kĩ thuật, về con đường nghề nghiệp, nhưng không có ai giải đáp.

Là một lập trình viên, các bạn cần học rất nhiều, nhưng không sách vở nào nói về cách tự học cho hiệu quả. Lập trình viên cần biết cách giao tiếp và làm việc nhóm, nhưng ít thầy cô nói cho các bạn biết điều này. Lập trình viên cần phải giỏi tiếng Anh, nhưng hầu như đi làm rồi các bạn mới tự nhận ra.

Không biết những điều này, bạn sẽ phải hứng chịu vô số gạch đá trên con đường nghề nghiệp. Do vậy, chúng ta cần những đầu sách định hướng nghề nghiệp và những kĩ năng phải có của người lập trình viên.

Tuy nhiên, đa phần sách cho dân IT hiện nay quá tập trung vào kĩ thuật và công nghệ (kĩ năng cứng), quên mất những kĩ năng mềm mà lập trình viên nên có. Những quyển sách trên cũng khá hàn lâm và khô cứng, khó tiếp thu.

Cuốn sách này không như thế! Vậy nó có gì hot?

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

Đây là cuốn sách duy nhất tập trung vào phần kỹ năng mềm mà mỗi lập trình viên cần có. Đi kèm với chúng những kỹ năng cứng được đúc kết qua kinh nghiệm bao năm làm việc của tác giả. Sách đảm bảo sẽ đưa bạn đọc từ mềm đến cứng.

Thay cho những chương sách dày cộm toàn chữ, nội dung sách được chia làm nhiều bài viết ngắn gọn, mỗi bài viết đề cập đến một khía cạnh khác nhau. Giọng văn ngắn gọn, hài hước dí dỏm, đọc không hề cứng nhắc như sách kỹ thuật mà lại rất dễ tiếp thu. Đoạn này không phải nhận xét của mình mà đó là nhận xét chung của khoảng 2000 bạn đọc ghé thăm blog mỗi ngày.

Ngành lập trình rất rộng lớn, không thể đề cập hết trong một cuốn sách. Do vậy, mình tập trung nhiều vào việc rèn luyện khả năng tự học và định hướng cho bạn đọc. Có kỹ năng tự học, có định hướng tốt, bạn sẽ dễ dàng sống sót và thăng tiến trong ngành này.

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

MỤC LỤC

TÓM TẮT NỘI DUNG

PHẦN 1 – KỸ NĂNG MỀM MỀM

VÀI LỜI KHUYÊN VÀ ĐỊNH HƯỚNG CHỌN TRƯỜNG CHO CÁC BẠN TRẺ
LẬP TRÌNH VIÊN CÓ CẦN HỌC ĐẠI HỌC HAY KHÔNG?
HAI SAI LẦM LỚN NHẤT TRONG QUÁ TRÌNH HỌC LẬP TRÌNH
ĐƯỢC GÌ MẤT GÌ KHI HỌC LẬP TRÌNH BẰNG TIẾNG VIỆT
TÔI ĐÃ HỌC TIẾNG ANH NHƯ THẾ NÀO
HỌC THUẬT TOÁN ĐỂ LÀM CÁI QUÁI GÌ?
NHỮNG ĐIỀU TRƯỜNG ĐẠI HỌC KHÔNG DẠY BẠN
TẠO ĐỘNG LỰC HỌC TẬP VÀ LÀM VIỆC – SỨC MẠNH CỦA THÓI QUEN
THỰC TRẠNG HỌC LẬP TRÌNH CỦA MỘT SỐ THANH NIÊN HIỆN NAY
THAY LỜI MUỐN NÓI – GỎI TỚI NHỮNG NGƯỜI THÂN YÊU CỦA MỖI LẬP
TRÌNH VIÊN
HỌC NGÔN NGỮ LẬP TRÌNH NÀO BÂY GIỜ
CÁCH TIẾP CẬN 1 NGÔN NGỮ/CÔNG NGHỆ MỚI
TOP CÁC “TRƯỜNG DẠY CODE” ONLINE CHO CÁC DEVELOPER
KỸ NĂNG CẦN CÓ CỦA MỘT WEB DEVELOPER
TỔNG QUAN VỀ LẬP TRÌNH ỨNG DỤNG DI ĐỘNG
MUÔN NẼO ĐƯỜNG TÌM VIỆC
CON ĐƯỜNG PHÁT TRIỂN SỰ NGHIỆP (CAREER PATH) CHO DEVELOPER
MẶT TỐI CỦA NGÀNH CÔNG NGHIỆP IT – PHẦN 1
TOP 18 SAI LẦM MÀ CÁC LẬP TRÌNH VIÊN “NON TRẺ” HAY MẮC PHẢI
ĐỪNG COI NGÔN NGỮ LẬP TRÌNH NHƯ TÔN GIÁO

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

LẬP TRÌNH VIÊN NÊN ĐỌC NHỮNG SÁCH GÌ?

SỰ THẬT ĐÁNG LÒNG: ĐÔI KHI CẮM ĐẦU NGỒI CODE LÀ CÁCH ... NGU NHẤT ĐỂ GIẢI QUYẾT VẤN ĐỀ

PHẦN 2 – KỸ NĂNG CỨNG CỨNG

XÓA MÙ VỀ AGILE VÀ SCRUM

TỔNG QUAN VỀ UI/UX TRONG NGÀNH LẬP TRÌNH

MỘT BUTTON TRỊ GIÁ 300 TRIỆU ĐÔ – CÁI NHÌN KHÁC VỀ GIAO DIỆN VÀ CHỨC NĂNG

LUẬN VỀ TECHNICAL DEBT – NỢ KIẾP NÀY, DUYÊN KIẾP TRƯỚC

GIẢI THÍCH ĐƠN GIẢN VỀ CI – CONTINUOUS INTEGRATION (TÍCH HỢP LIÊN TỤC)

SỰ KHÁC BIỆT GIỮA WEB SITE VÀ WEB APPLICATION

LUẬN VỀ COMMENT CODE (PHONG CÁCH KIỂM HIỆP)

NHẬP MÔN DESIGN PATTERN (PHONG CÁCH KIỂM HIỆP)

SOLID LÀ GÌ – ÁP DỤNG CÁC NGUYÊN LÝ SOLID ĐỂ TRỞ THÀNH LẬP TRÌNH VIÊN CODE “CỨNG”

SINGLE RESPONSIBILITY PRINCIPLE – NGUYÊN LÝ ĐƠN TRÁCH NHIỆM

OPEN/CLOSED PRINCIPLE – NGUYÊN LÝ ĐÓNG/MỞ

LISKOV SUBSTITUTION PRINCIPLE – NGUYÊN LÝ THAY THẾ LISKOV

INTERFACE SEGREGATION PRINCIPLE – NGUYÊN LÝ PHÂN TÁCH INTERFACE

DEPENDENCY INVERSION PRINCIPLE – NGUYÊN LÝ ĐẢO NGƯỢC DEPENDENCY

DEPENDENCY INJECTION VÀ INVERSION OF CONTROL

SAI LẦM HAY GẶP CỦA LẬP TRÌNH VIÊN MỚI VÀ NHỮNG MÁN KHÓE CỦA CÁC LẬP TRÌNH VIÊN VĨ ĐẠI

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

BÍ KÍP ĐỂ TRỞ THÀNH “CAO THỦ” TRONG VIỆC FIX BUG

CHUYỆN VỀ NHỮNG “Ổ GÀ” TRÊN CON ĐƯỜNG LẬP TRÌNH

ĐIỀU GÌ NGĂN CẢN BẠN ĐẠT CẢNH GIỚI TỐI CAO TRONG “CODE HỌC”?

PHẦN 3 – KÍ SỰ CODE DẠO

TẠM BIỆT ASWIG – ĐÔI DÒNG TÂM SỰ CỦA CHÀNG JUNIOR DEVELOPER

CHUYỆN ĐẦU NĂM – LẦN ĐẦU ĐI PHÒNG VẤN XIN VIỆC NƠI ĐẤT KHÁCH QUÊ NGƯỜI

NGÀY ĐẦU ĐI CODE DẠO NƠI ĐẤT KHÁCH QUÊ NGƯỜI

TẠM BIỆT LANCASTER ISS – TẠM KẾT THÚC KIẾP CODE DẠO NƠI XỨ NGƯỜI

LỜI CUỐI SÁCH

GIẢI THÍCH CÁC THUẬT NGỮ TRONG SÁCH

LINK ẢNH VÀ TÀI LIỆU THAM KHẢO

TÓM TẮT NỘI DUNG

Nội dung cuốn sách gồm 3 phần chính:

- Phần 1 tập trung vào những kỹ năng mềm và thái độ mỗi lập trình viên cần có trên ba quãng đường: Thuở còn ngồi ghế nhà trường, khi sắp ra trường và khi bắt đầu đi làm. Tuy vậy, các bạn sinh viên có thể đọc phần “làm việc” để hiểu thêm về công việc tương lai, cũng như các bạn đã đi làm có thể đọc phần “học hành” để biết và bổ sung những kỹ năng mình còn thiếu.
- Phần 2 đi sâu hơn về những kỹ thuật lập trình từ cơ bản đến nâng cao. Đa phần những kỹ thuật này không được dạy hoặc chỉ được dạy khá sơ sài ở nhiều trường đại học, dẫn đến việc sinh viên phải tự học, tự mò mẫm khi đi làm.
- Phần 3 là những mẩu chuyện và trải nghiệm nho nhỏ của chính tác giả trong quãng thời gian làm lập trình viên ở trong và ngoài nước.

Đối tượng chính của sách là các em lớp 12 sắp chọn ngành IT, các bạn sinh viên IT, những bạn lập trình viên vừa ra trường mới đi làm, và những bạn trẻ muốn tìm hiểu về ngành IT. Do vậy, sách không tập trung quá nhiều vào kỹ thuật (ngoại trừ phần 2 nặng về kỹ năng lập trình).

Bài viết trong sách sử dụng nhiều ví dụ sinh động, ngôn từ dễ hiểu, không hàn lâm những nên bạn đọc không có chuyên môn về IT cũng có thể thoải mái đọc và thưởng thức. Những từ ngữ thông dụng trong ngành IT sẽ được liệt kê phía cuối sách, giúp bạn đọc dễ tìm hiểu hơn.

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

PHẦN 1 – KỸ NĂNG MỀM MỀM

Đa phần các bạn sinh viên thường nghĩ rằng chỉ cần học giỏi, vững kỹ năng cứng (kỹ năng lập trình) thì sẽ dễ dàng kiếm được việc làm, thăng tiến. Đây là một suy nghĩ khá sai lầm, vì đôi khi kỹ năng mềm nhiều khi còn quan trọng hơn kỹ năng cứng rất nhiều lần.

Nếu bạn code giỏi nhưng không biết giao tiếp với trưởng nhóm và các thành viên khác, bạn sẽ không thể truyền đạt ý kiến của mình hay lãnh đạo. Nếu bạn lập trình tốt nhưng không rành tiếng Anh, không biết tự học thì kiến thức của bạn sẽ rất nhanh hết thời, làm bạn bị tụt hậu. Hoặc giả bạn có giỏi đến mấy nhưng nếu cứ mang thái độ “mình là sinh trường A, B danh giá, giỏi hơn hẳn bọn kia!” đi xin việc, bạn sẽ rớt ngay từ vòng gửi xe, à không, gửi nón.

Do vậy, mình dành ra phần đầu cuốn sách để tập trung vào những kỹ năng mềm mà lập trình viên cần có, nên có và phải có để trở thành một lập trình viên (developer) chuyên nghiệp.

Giai đoạn 1 – Học hành

Đây là giai đoạn bạn cần tập trung học các kiến thức nền tảng trong trường, rèn luyện khả năng tự học, tiếng Anh v...v. Các bài viết trong phần này sẽ mang tính định hướng, đồng thời đề cập tới những kỹ năng nói trên.

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

VÀI LỜI KHUYÊN VÀ ĐỊNH HƯỚNG CHỌN TRƯỜNG CHO CÁC BẠN TRẺ

Chọn trường đại học và chọn ngành học là một ngưỡng cửa khá quan trọng của cuộc đời. Có lẽ đa phần bạn đọc của sách là sinh viên đại học, hoặc đã đi làm nên chắc sẽ không cần đọc bài này. Tuy nhiên, hãy đưa nó cho em/cháu bạn hoặc phụ huynh của các em. Bài viết sẽ giúp họ có cái nhìn đúng hơn trong việc chọn trường, giúp các em hiểu hơn về công việc mình sẽ làm trong tương lai.

Lời khuyên đầu: Học trường vừa sức

Lời khuyên đầu tiên mình muốn gửi tới các bạn và các em là: Chọn trường vừa sức mình. Vừa sức ở đây không chỉ có nghĩa là vừa sức đầu, mà còn có nghĩa là vừa sức học và cạnh tranh.

Tại sao lại chọn trường vừa sức mà không phải là trường nổi tiếng? Theo lẽ thường, chất lượng dạy và học ở của các trường nổi tiếng này khá cao, tấm bằng đại học danh tiếng cũng rất có ích khi bạn vừa ra trường xin việc hoặc muốn tiếp tục học lên cao.

Tuy nhiên, ở các trường này, do chất lượng đầu vào cao nên bạn sẽ phải học hành và cạnh tranh với những bạn bè giỏi hơn (còn được gọi dưới cái tên thiên tài hay quái vật). Nếu không đủ giỏi, việc cạnh tranh với những thành phần này dễ làm bạn nản lòng thoái chí. Chưa kể, do các giáo viên đã quen với việc dạy dỗ học sinh thông minh, có thể họ sẽ giảng giải với tốc độ nhanh hơn, khó hiểu hơn, làm bạn khó theo kịp.

Ngoài ra, vào những trường giỏi, bạn rất khó để vào top đầu lớp hoặc gây ấn tượng với giáo viên (vì học sinh giỏi nhiều quá rồi). Ngày xưa, mình cũng đậu đại học BK HCM nhưng không học, một phần là do FPT có học bổng 70%, một phần là do mình không muốn bỏ quá nhiều công sức vào việc cạnh tranh học tập. Vào FPT, mình dễ dàng đứng đầu lớp, được nhiều giáo viên thương và để ý. Nhờ vậy, mình dễ dàng xin thư giới thiệu của họ khi làm đơn du học.

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

Lời khuyên thứ hai: Tự học đi, không ai dạy bạn đâu!

Lời khuyên thứ hai là: Kiến thức ở trong trường không đủ để bạn xin việc đi làm đâu¹. Do đó hãy bắt đầu rèn luyện kỹ năng tự học và tìm hiểu từ bây giờ đi. Những kiến thức bạn có được khi học đại học chỉ là nền tảng thôi, khó mà áp dụng ngay vào công việc! Nếu chỉ biết những gì được dạy mà không biết tự mày mò học thêm, bạn sẽ gặp khá nhiều khó khăn khi ôm mớ nền tảng đó đi xin việc đấy!

Nhiều bạn sinh viên đi học được một, hai năm thì bắt đầu rơi vào tình trạng mất phương hướng vì cảm thấy những thứ mình học quá vô dụng, chẳng làm được gì. Thay vì chờ được thầy cô dạy, hãy tận dụng những kiến thức nền tảng đã có để tự học, sau đó áp dụng những thứ vừa học để tạo ra một sản phẩm gì đó, bạn sẽ thấy thích học ngay thôi. Còn nữa, nhớ phải tập trung trau dồi tiếng Anh nhé². Học IT mà không giỏi tiếng Anh thì khó phát triển lắm đấy!

Học IT xong thì ra làm gì?

Ở Việt Nam, hầu như mọi người chỉ biết ngành IT (Information Technology – Công nghệ thông tin), chứ không biết tường tận ngành đó làm những gì. Do đó, mình sẽ giải thích một số chuyên ngành của ngành IT, về những thứ bạn sẽ học cũng như công việc bạn sẽ làm sau khi ra trường.

Hiện tại ngành IT có một số chuyên ngành sau:

- **Khoa học máy tính (Computer Science):** Bạn sẽ học các thứ liên quan tới cách thức máy tính hoạt động. Theo như tên gọi “Khoa học”, chuyên ngành này thường nặng về nghiên cứu.
- **Kỹ nghệ phần mềm (Software Engineering):** Ngành này cũng học một số môn tương tự như CS. Tuy nhiên, chuyên ngành này thiên về thực tế và xây dựng phần mềm nên bạn được học thêm 1 số ngành như: Quy trình phát triển phần mềm, Kiểm thử phần mềm. Học ngành này bạn có thể viết ứng dụng,

¹ Đọc kĩ hơn trong bài viết: Những điều trường đại học không dạy bạn”

² Mình có chia sẻ kinh nghiệm học và thi trong bài “Tôi đã học tiếng Anh như thế nào”

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

viết website hoặc xây dựng một hệ thống. Sinh viên tốt nghiệp cả hai ngành CS và SE đều có thể làm lập trình viên (developer).

- **Hệ thống thông tin (Information System):** Ngành này thiên về phân tích thiết kế hệ thống dựa theo yêu cầu của khách hàng. Bạn sẽ phải học một số môn liên quan tới Thương mại điện tử, cách thức các doanh nghiệp hoạt động. Khi ra trường bạn có thể làm ở vị trí Business Analyst (BA).
- **Hệ thống nhúng (Embedded System):** Ngành này tập trung vào việc xử lý tín hiệu số, thiết kế mạch điện, chip điện tử và linh kiện. Khi ra trường, bạn cũng là lập trình viên, nhưng lập trình cho các thiết bị hoặc mạch điện. Ngành này hơi khó và khô khan hơn ngành SE, nhưng lương trung bình cao hơn một chút.
- **Lập trình mạng (Network Engineering):** Ngành này dạy về cơ sở hạ tầng mạng, cách lắp đặt hệ thống, v...v. Mấy bác tốt nghiệp ngành này là những người cài win dạo, bấm cáp dạo, sửa modem, quản lý server, thường gọi là IT Helpdesk. Họ là những người hùng thầm lặng, giúp hệ thống hoạt động trơn tru.
- **An toàn thông tin (Information Security):** Ngành này tập trung về bảo mật, bạn sẽ được học về kiến trúc hệ thống, mã hóa, bảo mật, những phương thức hack và cách phòng chống. Ngành này phù hợp với những bạn hâm mộ các anh hacker. Ra trường, bạn có thể làm hacker mũ trắng hoặc chuyên viên bảo mật cho các công ty.

Có một số môn như Hệ điều hành, Mạng máy tính, Mã máy, Thuật toán, Cấu trúc dữ liệu.... mà sinh viên chuyên ngành nào cũng phải học. Giữa các trường đại học, chương trình học của các chuyên ngành này sẽ có đôi chút khác biệt.

Tóm tắt:

- Nên chọn trường vừa sức để bạn có thể nằm trong top đầu lớp
- Cần rèn luyện khả năng tự học. Trong ngành này, tự học là chính, kiến thức trong nhà trường là không đủ
- Tùy vào ngành học mà sinh viên IT ra trường có thể làm rất nhiều nghề: lập trình viên, quản trị mạng, an toàn thông tin,...

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

HAI SAI LẦM LỚN NHẤT TRONG QUÁ TRÌNH HỌC LẬP TRÌNH

Bài viết này nói về hai sai lầm lớn nhất trong quá trình học lập trình. Qua trao đổi với các bạn sinh viên, mình nhận thấy có khá nhiều bạn sinh viên mắc phải những sai lầm này (cá nhân mình hồi năm nhất năm hai cũng thế). Do đó, bài viết này sẽ cảnh tỉnh một số bạn, đồng thời chia sẻ chút kinh nghiệm để tránh các bạn đi theo vết xe đổ của mình ngày trước.

Câu nói hay gặp – trường em không dạy A, B, C ...

Dưới đây là một mẫu đối thoại giữa mình và một bạn sinh viên (giấu tên)

- Bạn: Anh ơi, học C với C++ ra trường thì làm được gì anh?
- Mình: Làm hệ thống nhúng hoặc game em nhé, lương khủng lắm đấy.
- Bạn: Em thích làm Web hoặc làm app di động cơ, C++ làm được không anh?
- Mình: Không em nhé, muốn làm web thì học HTML/CSS/JS. Sau đó có thể chuyển qua làm hybrid app mobile ³, hoặc học Android để viết app.
- Bạn: Mấy cái đó trường em không dạy anh ơi!!!
- Mình: ...

Một câu mình nói mình hay được nghe các bạn nói là: trường em chỉ dạy C, trường em chỉ dạy Java, mấy thầy cô không dạy HTML hay làm Web... Mình đã từng nói ở đầu sách, đại học chỉ cho bạn các kiến thức nền tảng về lập trình. Họ sẽ không dạy bạn cách code như thế nào, cách làm việc, cách sử dụng một ngôn ngữ hoặc framework ra sao, mà bạn sẽ phải tự dạy mình!!.

Thái độ trường không dạy nên không biết là một thái độ học tập cực kỳ sai lầm. Vấn đề không phải người ta dạy cho bạn cái gì, mà là bạn có thể học được cái gì! Thái độ ngồi chờ sung rụng, có người dạy mới học này sẽ cản trở bạn trên con đường tìm hiểu cái mới, tự cập nhật

³ Xem thêm trong bài “Tổng quan về lập trình ứng dụng di động”

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

kiến thức cho bản thân. Nếu giữ thái độ này, kiến thức của bạn sẽ nhanh chóng lạc hậu, lỗi thời, gây ra nhiều khó khăn trên con đường thăng tiến của bạn.

Học tập thế nào cho đúng??

Việc học phải mang tính chất chủ động chứ không phải là bị động. Bạn phải tự tìm cái cần học và tự sắp xếp thời gian để học. Bạn nào kiến thức vững, đủ kiên nhẫn để tự học thì có thể tải ebook tiếng Anh hoặc tìm nguồn tự học trên mạng. Bạn nào kiến thức còn yếu thì có thể ra trung tâm để có người kèm cặp.

Nhà tuyển dụng chỉ cần biết bạn có kiến thức hay không và có làm được việc hay không. Họ không quan tâm là kiến thức đó bạn có được từ nhà trường, từ trung tâm hay do tự học. Thứ duy nhất bạn phải nhớ là: thầy cô hay trung tâm cũng không thể vá lỗ hổng kiến thức hay dạy cho bạn tường tận được, mà chính bạn mới là người nỗ lực hấp thu, biến kiến thức của họ thành kiến thức của mình.

Chưa kể, chương trình học ở các trường bây giờ... cũ xì, quanh đi quẩn lại chỉ có WinForm, WebForm, Java Servlet... . Nếu cứ “há miệng chờ sung, dạy gì học nấy”, bạn sẽ không có đủ kỹ năng cần thiết để xin việc khi ra trường đâu nhé!

Ngoài ra, đừng nghĩ rằng chỉ học một lần cho biết là xong, nguy hiểm lắm! Công nghệ liên tục thay đổi, bạn cũng phải thường xuyên cập nhật kiến thức bản thân. Trước đây mình từng có khoảng 2 năm kinh nghiệm lập trình C#. Đầu năm nay, lúc mình xem lại thì công nghệ đã được cập nhật, làm cho kiến thức cũ của mình lỗi thời hết cả! Thay vì than trời trách đất, mình đành phải tự học để cập nhật kiến thức mới thôi.

Sai lầm thứ hai: Cẩn thận, chưa chắc học nhiều/xem nhiều là sẽ giỏi!!

Người Việt chúng ta có thói quen ghét ai ghét cả đường đi lối về. Khi đã tin tưởng hay thần tượng ai đó thì nó nói gì cũng tin; khi đã ghét thằng nào thì nó nói gì cũng sai, cũng nhầm nhứ. Thái độ này dễ làm bạn tiếp nhận sai tiếp nhận thông tin sai cách!

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

Mình hay đọc sách Tony Buổi Sáng, có những bài viết về cách nhìn cuộc sống khá hay. Thay vì hâm mộ, nuốt từng câu từng chữ của “dượng”, vẫn có những đoạn văn chém gió, những cách nhìn mà mình không đồng tình. Tuy vậy, mình vẫn chắt lọc những điều hay, những điều tác giả muốn gửi gắm, còn mấy vấn đề mình không đồng tình thì bỏ qua.

Tương tự, các bạn nên đọc sách, nghe thầy cô giảng nhưng đừng nên tin tưởng hoàn toàn những gì được nghe. Hãy tự hỏi xem: Thầy cô hay sách nói như vậy đúng hay sai, có chứng cứ gì không? Cuộc sống có câu “Không có gì là vĩnh cửu”, có thể bây giờ mình cho điều đó là đúng, nhưng trong tương lại điều đó lại sai thì sao?

Đừng tin toàn bộ những gì sách nói, cũng đừng nuốt từng câu từng lời của thầy cô hay tác giả. Nghe người khác nói cái gì cũng phải nghi ngờ và kiểm chứng. Hãy xem tác giả là một thanh niên đang chém gió với mình thông qua sách, cái gì đúng thì gật gù đồng ý, cái gì sai thì phản bác lại ngay.

Ngoài việc học nhiều, bạn còn phải biết cách lọc bỏ, chọn lựa những thông tin có ích cho bản. Những gì hay thì hãy ghi nhớ và học theo; những gì nhảm nhí thì cứ bỏ qua, coi như nó không tồn tại. Nói một cách dân dã là phải biết cách “gạn đục khơi trong” từ vô số nguồn kiến thức.

Kết luận

Sửa được hai sai lầm về thái độ học tập bị động và chọn lọc kiến thức, bạn sẽ thấy mình trở nên vô cùng tự tin. Công nghệ A/B không có trong chương trình học? Chả sao, chỉ cần tự học vài buổi là xong! Càng học nhiều, bạn sẽ càng thấy việc học cái mới trở nên rất dễ dàng và nhanh chóng.

Hi vọng, sau bài viết này, mình sẽ không còn phải nghe câu “em không biết cái ABC này, trường với thầy cô không dạy” nữa. Thay vào đó, mình hi vọng sẽ được nghe câu: “Em đang tự tìm hiểu cái ABC, anh chỉ em một số nguồn học và những điều cần lưu ý nha”.

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

Tóm tắt:

- Đừng trong chờ vào việc nhà trường sẽ dạy cho bạn kiến thức để làm việc. Chịu khó tự học càng sớm càng tốt.
- Đọc nhiều, học nhiều là tốt, nhưng phải biết cách loại bỏ những thứ vô bổ, giữ lại những điều có ích.

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

Giai đoạn 2 – Ra đời

Đây là giai đoạn bạn gần ra trường, sắp đi làm. Ở giai đoạn này, do đã có kiến thức nền tảng vững từ những môn học ở trường, bạn cần tập trung tìm hiểu thêm về ngành nghề, đồng thời tự trang bị những kỹ năng cần có để xin việc.

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

HỌC NGÔN NGỮ LẬP TRÌNH NÀO BÂY GIỜ?

Đây một câu hỏi mà mình thường nhận được từ các em sinh viên mới ra trường, mới vào đại học, hoặc chưa biết gì về lập trình: “Giờ mình nên học ngôn ngữ lập trình nào đây?”.

Nghe đơn giản, nhưng đây là một câu hỏi có độ khó khá cao, sánh ngang với câu “Em nên làm nghề gì, học đại học nào?” của các em học sinh cấp 3. Trong phạm vi bài viết này, mình sẽ đưa ra một số dữ liệu tham khảo và lời khuyên cá nhân.

Trước khi hỏi câu này, hãy tự hỏi : Mình muốn học lập trình để làm gì?

Khi được hỏi “Giờ mình nên học ngôn ngữ lập trình nào đây?”, mình luôn hỏi lại câu này “Bạn/Em muốn học lập trình để làm gì?”. Trả lời được câu hỏi này, bạn đã xác định được 50% ngôn ngữ mình cần học. Dưới đây là một số câu trả lời mình hay nhận được.

1. Em vừa ra trường, trường chỉ dạy C, C++, ... giờ em cần học ngôn ngữ gì để dễ kiếm việc làm, lương cao?

Thị trường việc làm IT hiện tại khá rộng, tạm chia làm 3 mảng: embedded (lập trình nhúng), web và mobile. Thị phần mảng Game khá nhỏ nên mình không nhắc đến.

- **Mảng embedded:** yêu cầu khá cao về trình độ, sử dụng ngôn ngữ lập trình C, C++, có thể dùng Java. Nếu bạn là lập trình viên C++ cứng, mức lương rất khá và mức độ cạnh tranh cũng ko nhiều.
- **Mảng mobile:** Chiếm thị phần cao nhất vẫn là app cho Android viết bằng Java, tiếp theo là app cho IOS, viết bằng Objective-C⁴. Java là một ngôn ngữ khá dễ học, độ phổ biến cũng cao, ứng dụng rộng. Với kiến thức Java bạn cũng có thể chuyển qua mảng Web.
- **Mảng web:** Để có thể trở thành lập trình viên Web, bạn phải biết lập trình front-end (Dùng HTML/CSS và ngôn ngữ

⁴ Xem kĩ hơn trong “Tổng quan về lập trình ứng dụng di động”

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

JavaScript). Sau đó học một ngôn ngữ lập trình back end (C#, Java, PHP)⁵.

Nếu muốn học để kiếm tiền, hãy xác định mình muốn làm mảng công việc nào, sau đó tìm các khảo sát trên mạng và nghiên cứu mức lương trung bình của developer ngôn ngữ đó.

Hiện tại, có khá nhiều PHP developer nên lương trung bình của PHP developer hơi thấp so với số còn lại. Ngoài ra, một số ngôn ngữ như Rails, Python,... ít người học, developer hiếm nên biết các ngôn ngữ này sẽ có thu nhập khá hơn.

2. Mình muốn làm website hoặc ứng dụng cho người nhà, bản thân v....v.

Đây là câu trả lời mình nhận được từ một số bạn học tài chính ngân hàng, kinh tế,... Nếu bạn muốn làm một ứng dụng di động, Java là lựa chọn tốt nhất. Để tạo website, hiện tại có rất nhiều hướng dẫn tạo website bằng Joomla, Drupal, Wix,... mà ko cần kiến thức lập trình. Các bạn có thể học thêm PHP để có thể tùy biến, thêm tính năng cho trang web.

Lựa chọn thật ra không quan trọng, học một ngôn ngữ mới là chuyện đơn giản

Đọc tới đây, hẳn nhiều bạn sẽ bảo rằng mình nổ, hoặc nghĩ rằng “dám chắc thằng này không phải coder, phán như thánh”. Trước khi ném đá, hãy bình tĩnh nghe mình giải thích và trình bày.

Mình cũng từng là sinh viên IT như các bạn. Môn đầu tiên về lập trình mình được học khi vào đại học là: “Cơ bản lập trình với C”. Mình từng điên đầu với khai báo biến, tách hàm, điều kiện, vòng lặp, IO.... Môn tiếp theo là “Lập trình hướng đối tượng với C++”. Phải thú thật C++ không phải là ngôn ngữ phù hợp để học hướng đối tượng. Mình từng nhầm lẫn trước các khái niệm “tính bao đóng, tính kế thừa”.

Do đó, bản thân mình cũng biết sự khó khăn gặp phải khi học một ngôn ngữ. Tuy vậy, mình vẫn khẳng định học một ngôn ngữ mới là chuyện đơn giản.

⁵ Xem kĩ hơn trong “Kỹ năng cần có của Web Developer”

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

Vì sao? Hãy tự xem lại kiến thức lập trình bạn có được khi vừa ra trường:

- Học qua 1,2 ngôn ngữ gì đó
- Cấu trúc dữ liệu và thuật toán
- Thiết kế, truy vấn cơ sở dữ liệu
- Design pattern (Có thể)
- Khả năng design

Khi mới tiếp cận lập trình, chúng ta cảm thấy khó khăn vì phải làm quen với vô số khái niệm mới. Tuy nhiên, khi đã có kiến thức cơ bản, việc tiếp cận ngôn ngữ mới trở nên rất dễ dàng. Chúng ta học gì khi học một ngôn ngữ lập trình mới? Đây là câu trả lời:

- Cách khai báo hàm, biến
- Cách khai báo vòng lặp, điều kiện if/else
- Các kiểu cấu trúc dữ liệu: list, set, tuple, ...
- IO, multi-thread, delegate, event
- IDE phù hợp, cách build, debug
- Các framework, cách sử dụng,

Nếu bạn đã biết cách viết for, if/else, while ... trong Java, khi chuyển qua học C# hoặc javascript, cấu trúc hàm for, if/else... vẫn giữ nguyên. Kiến thức của bạn được kế thừa từ ngôn ngữ lập trình trước, do đó việc học sẽ diễn ra nhanh hơn. Hoặc khi bạn đã rõ cơ chế làm việc của ASP.NET RestAPI, việc học cách xây dựng RestAPI bằng Spring của Java cũng không quá khác biệt.

Mình từng tự học Python mất 1 tuần, và học framework Django mất khoảng 2 tuần nữa. Lý do mình học nhanh vậy là vì:

- Mình đã có kiến thức cơ bản về lập trình (class, data structure)
- Mình biết những gì mình cần học. Khi mới lập trình, bạn không biết mình cần học gì. Tuy nhiên nếu đã có kiến thức nói chung về lập trình, bạn sẽ biết mình tập trung học những gì, điều này tiết kiệm rất nhiều thời gian.
- Mình biết là mình làm được. Khi mình hỏi bạn bè chung ngành “Học một ngôn ngữ mới mất bao lâu”, hầu hết đều trả lời “một

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

tháng hoặc hơn”. Vì thấy tốn nhiều thời gian và khó khăn như vậy nên hầu như họ rất “ngại” học ngôn ngữ mới.

Đừng sợ mình sẽ chọn nhầm ngôn ngữ, cứ học đi. Việc học một ngôn ngữ lập trình mới khi bạn đó có kiến thức cơ sở khá đơn giản, không hề khó khăn và mất thời gian như bạn nghĩ. Thêm vào đó, việc biết nhiều ngôn ngữ sẽ giúp bạn có lợi thế hơn khi xin việc.

Lời khuyên của bản thân Hoàng

Dưới đây là một số lời khuyên của mình, dựa theo kinh nghiệm cá nhân (Mình chỉ có kinh nghiệm mảng web và mobile, nên các lời khuyên có thể sẽ không áp dụng được cho mảng embedded system):

...

Tóm tắt

- Trước khi hỏi “Cần học ngôn ngữ gì?”, hãy tự trả lời “Học lập trình để làm gì?”
- Đừng lo chọn sai ngôn ngữ để học, học một ngôn ngữ mới rất đơn giản
- Đừng chạy theo công nghệ, hãy tập trung vào kiến thức cơ bản và những thứ lâu bền
- Nếu quá dư thời gian, hãy tập trung vào học JavaScript

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

MUÔN Nẻo ĐƯỜNG TÌM VIỆC

Không như các ngành nghề khác, ngành lập trình là một ngành khá dễ xin việc ở Việt Nam. Chỉ cần có khả năng code kha khá, các bạn sinh viên có thể dễ dàng xin việc với mức lương tạm ổn, không cần phải quen biết, lót tay hay con ông cháu cha gì cả.

Bài viết này chia sẻ kinh nghiệm mình có được trong quá trình xin việc, từ lúc viết CV cho đến lúc phỏng vấn. Hi vọng chúng sẽ có ích cho bạn trên bước đường tìm việc.

Phần 1: Tìm việc và viết CV

Tìm việc ở đâu?

Các trang web dưới đây là những trang web về việc làm lớn nhất Việt Nam hiện tại. Đây là những kênh trung gian giữa các nhà tuyển dụng và người tìm việc, do đó mình khuyên các bạn đang tìm việc hãy tạo một CV online cho mỗi trang này:

- IT Việc: <https://itviec.com/>
- LinkedIn: <https://www.linkedin.com/>
- Vietnamworks: <http://topit.vietnamworks.com/>
- Jobtown: <https://jobtown.co/>
- Một số group lập trình trên facebook cũng thường có các mẫu tin tuyển dụng.

Lâu lâu vẫn có nhận được điện thoại mời phỏng vấn của nhân sự các công ty. Mình không cần nộp CV hay ứng tuyển vì họ đã xem CV trên mạng rồi. Cá nhân mình khuyến khích các bạn nên trau chuốt CV trên LinkedIn, chức năng connect của trang này cho phép bạn kết nối với nhiều người (đồng nghiệp, cấp trên), mở rộng mạng lưới nghề nghiệp của bạn.

Viết và gửi CV

1. Nội dung

Do đã có vô số các bài viết hướng dẫn viết CV trên mạng rồi, mình sẽ không nhắc lại. Nếu bạn chưa biết trình bày CV như thế nào, cần những

LẬP TRÌNH VIÊN ĐẦU PHẢI CHỈ BIẾT CODE

thông tin gì, hãy tìm một CV mẫu cho vị trí của mình (vd bạn là junior developer hãy xem CV mẫu của 1 junior dev, tương tự với junior QC), chỉnh sửa lại cho phù hợp. Format CV của linkedin cũng khá ổn; chỉ cần bê các phần từ trên linkedin về, chỉnh sửa một chút là bạn đã có một CV khá chuẩn.

Một điều cần lưu ý: Hãy chỉnh sửa CV cho phù hợp với vị trí bạn ứng tuyển. Giả sử bạn biết cả C#, Java, Python, nhưng công việc bạn ứng tuyển đòi hỏi C# MVC, hãy để các kỹ năng và dự án liên quan tới C# lên đầu trong CV.

2. Cách trình bày

CV không cần phải đẹp lộng lẫy nhưng phải rõ ràng và ngắn gọn. Cỡ chữ nên phù hợp là 12-14, sử dụng font cơ bản như Times New Roman hoặc Arial. Các phần đề mục nên viết rõ ràng, in đậm, nhìn lướt qua có thể đọc được. Thường thường bên tuyển dụng cũng không quá bắt bẻ về mặt hình thức, nhưng các bạn nên chịu khó canh thẳng hàng thẳng lối, đồng thời soát kỹ các lỗi chính tả. Những điều nhỏ nhặt này sẽ thể hiện tính chuyên nghiệp và sự cẩn thận của bạn.

Các bạn cũng nên xuất file CV ra dưới dạng PDF để máy nào cũng mở được. CV nên đặt tên theo format “[Tên vị trí] Tên bạn”, giúp nhà tuyển dụng dễ đọc và phân loại (Hoặc bạn đặt tên theo format mà nhà tuyển dụng đưa ra trong quảng cáo tuyển dụng).

3. Một số sai lầm hay gặp

- **Ảo tưởng sức mạnh:** Không biết vô tình hay cố ý mà trong phần kỹ năng, nhiều bạn tự đánh giá trình độ của mình là Intermediate, Expert, hoặc Master, dù chỉ mới ra trường, chỉ mới làm 1,2 cái đồ án chứ chưa làm dự án lớn nào. Nếu đã từng làm qua một công nghệ nào, các bạn nên ghi thời gian đã tiếp xúc và mức độ nắm vững công nghệ đó là đủ. Trình độ bạn ở mức nào thì người phỏng vấn sẽ tự xác định, bạn có nói mình là Master họ cũng không tin đâu.
- **Gây chú ý không đúng cách:** Dùng font màu mè lạ mắt để đập vào mắt nhà tuyển dụng. Cách này thường gây tác dụng ngược. Họ có thể quảng CV của bạn vào sọt rác không thương tiếc.

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

- **Chém gió:** Mình từng gặp trường hợp có bạn chém gió về số năm đi làm và công nghệ mình biết trong CV. Tới lúc phỏng vấn, bị vặn hỏi thì bạn bảo là: mình làm part time, công nghệ ABC gì đó mình hiểu nhưng chưa làm bao giờ ... Nhiều đó là quá đủ để rớt đài rồi nhỉ? Hãy nhớ rằng mục đích của CV chỉ là giúp bạn qua được vòng gửi xe, được mời phỏng vấn thôi, chứ không giúp bạn có được việc làm ngay đâu nhé.

Phần 2: Tham gia phỏng vấn

...

Tóm tắt

- Ngành IT rất dễ tìm việc, một số kênh thường dùng: itviec, linkedin, vietnamwork.
- CV là thứ đầu tiên gây ấn tượng với nhà tuyển dụng. Hãy trình bày rõ ràng, nội dung chân thật, không thổi phồng khả năng của bản thân.
- Trước khi phỏng vấn, hãy chịu khó tìm hiểu công ty, tìm hiểu thị trường và ôn lại kiến thức.
- Luôn tỏ thái độ thân thiện hòa nhã khi phỏng vấn. Sau khi kết thúc phỏng vấn, nhớ gửi email cảm ơn.

LẬP TRÌNH VIÊN ĐÂU PHẢI CHỈ BIẾT CODE

Phần 3 – Làm việc

Sau khi tốt nghiệp ra trường và vượt qua vòng phỏng vấn gian khổ, bạn sẽ được nhận vào một công ty phần mềm. Bắt đầu trong một môi trường mới, bạn sẽ khá bối ngỡ khi lần đầu tiếp xúc với dự án thực tế, tuân theo qui trình, làm việc theo nhóm. Hẳn bạn cũng sẽ có vài câu hỏi về nghề nghiệp, về tương lai. Những câu hỏi đó sẽ được trả lời trong phần này.

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

CON ĐƯỜNG PHÁT TRIỂN SỰ NGHIỆP (CAREER PATH) CHO DEVELOPER

Các bạn sinh viên còn đang học hoặc mới ra trường sẽ khó hình dung được về những vị trí và chức danh trong ngành lập trình. Bài viết này sẽ giải đáp một số thắc mắc của các bạn như:

- Mới đi làm em có chức danh gì, công việc thế nào?
- Code lâu thì lên được chức gì, lương có cao không?
- Em chỉ thích code thôi, không thích làm trưởng nhóm, em nên định hướng thế nào.

Hiểu rõ con đường nghề nghiệp của ngành developer, các bạn sẽ dễ định hình phát triển tương lai của bản thân, cũng như dồn sức vào con đường mình đã chọn.

Mình chỉ liệt kê con đường nghề nghiệp của một developer vì bản thân mình cũng là developer. Con đường của 1 tester (QA engineer) cũng có một số chức danh tương tự nhưng lên cao sẽ khác. Các chức vụ sẽ được miêu tả theo thứ tự từ thấp lên cao nhé.

Lưu ý: Mức lương đề cập trong bài dựa theo mức lương và quảng cáo tuyển dụng của các công ty ở TP. HCM, ở các tỉnh thành khác sẽ có một số chênh lệch.

Fresher/Junior Developer

Các bạn sinh viên đi thực tập hoặc mới ra trường thường được chức danh này. Kinh nghiệm của Junior Developer thường vào khoảng 6 tháng – 1 năm. Mức lương thì tùy vào khả năng của bạn, thường dao động trong khoảng 300-500\$.

Do chưa có kinh nghiệm nên fresher/junior thường được các công ty đào tạo lại, vì vậy khi phỏng vấn vị trí fresher các công ty thường chỉ chú trọng khả năng suy nghĩ logic, kiến thức lập trình cơ bản và tiềm năng phát triển của bạn. Cá nhân mình thấy chương trình đào tạo Fresher của FSOFTE cũng khá tốt, có dạy nhiều thứ mà bạn sẽ tiếp xúc khi làm việc.

Do chưa có kinh nghiệm nên mọi người thường không đòi hỏi ở bạn quá cao. Công việc của một junior thường là tìm hiểu dự án hiện tại,

LẬP TRÌNH VIÊN ĐẦU PHẢI CHỈ BIẾT CODE

code các module đơn giản, fix bug với sự trợ giúp và review của senior. Ở giai đoạn junior, các bạn hãy cố gắng tranh thủ học cách code, cách thức làm việc và kinh nghiệm của các bác senior đi trước.

Developer

Code được một thời gian khoảng 1-2 năm, các bạn sẽ được gọi là Developer (Nhiều bác lên thẳng vị trí Team Leader hoặc Senior tùy vào công ty). Ở giai đoạn này, bạn đã làm qua một số project, khá rành về một số công nghệ. Mức lương của developer vào khoảng 600-900\$.

Phỏng vấn cho developer dĩ nhiên là khó hơn junior. Người phỏng vấn sẽ hỏi bạn về những dự án bạn đã làm, các khó khăn bạn đã gặp phải và cách giải quyết? Ngoài ra, buổi phỏng vấn sẽ tập trung vào những công nghệ bạn đã ghi trong CV. Vì developer đã có kinh nghiệm, khi đi làm các bạn sẽ không còn “được” các anh senior kèm cặp, và cũng khó mà lấy danh nghĩa junior để hỏi, nhờ vả hay mắc lỗi nữa.

Ở giai đoạn này, bạn đã được code một số module phức tạp hơn, tham gia họp, code review, thảo luận với khách hàng v...v. Đây là giai đoạn để bạn dồn nén kiến thức, kinh nghiệm và gây dựng danh tiếng để lên nấc tiếp theo trong bậc thang nghề nghiệp.

Lý thuyết là vậy nhưng thực tế, thuở làm ở FSOFT mình ở vị trí junior được khoảng một tháng rồi nhảy vào làm các công việc của developer, nhận cả những việc khó chứ không đùn đẩy gì, nhờ vậy cũng mình học hỏi được khá nhiều.

Quản lý hay kỹ thuật?

Ở giai đoạn sau, bạn đã có thể xác định con đường cho mình. Nếu muốn tập trung vào code và kỹ thuật, bạn có thể đi theo hướng kỹ thuật: Senior Developer => Technical Lead => Software Architecture. Nếu muốn làm việc với quy trình và con người, bạn nên đi theo hướng quản lý: Team Lead => Project Manager => Manager.

Ở giai đoạn đầu, lần ranh giữa 2 con đường này khá mờ nhạt, nhưng càng lên cao lại càng trở nên rõ ràng. Các bạn có thể xem bảng tóm tắt sau:

LẬP TRÌNH VIÊN ĐỀU PHẢI CHỈ BIẾT CODE

Hướng quản lý (Management)	Hướng kỹ thuật (Technical)
Team Leader Bạn trở thành trưởng một nhóm nho nhỏ khoảng 3-6 thành viên. Ngoài trừ code, bạn còn phải hợp hành với cấp trên, báo cáo với khách hàng, quản lý cấp dưới. Ở giai đoạn này, bạn sẽ dần học thêm một số kỹ năng lãnh đạo, kỹ năng quản lý v...v. Ở một số công ty nhỏ, developer lâu năm và có kinh nghiệm sẽ lên team leader. Bạn vẫn còn khá nhiều thời gian code, code giỏi có thể sẽ làm thành viên trong team tôn trọng hơn. Mức lương cho team leader thường vào khoảng 1000-1500\$	Senior Developer Sau một thời gian làm việc, bạn nắm vững, hiểu sâu và rộng nhiều công nghệ và quy trình. Ở vị trí này, ngoài trừ khả năng code “thần thánh”, bạn còn phải biết đưa ra design và solution. Ngoài ra, bạn còn phải hướng dẫn chỉ bảo các em junior mới vào, cũng như tham gia code review v...v. Đôi khi senior dev cũng kiêm luôn vị trí team leader, do đó bạn cũng cần một chút kỹ năng diễn đạt và lãnh đạo. Mức lương cho Senior Developer cũng vào khoảng 1000-1500\$ (hoặc hơn).
Project Manager Lên đến vị trí này, bạn sẽ có rất ít hoặc hầu như không có thời gian code. Đa phần thời gian của bạn dùng để đọc báo, lướt voz, lướt webtretho Đều đấy, công việc chính của bạn bây giờ là quản lý cấp dưới, báo cáo với khách hàng và cấp trên về tiến độ dự án, lâu lâu bạn còn phải đi phỏng vấn một số ứng viên	Technical Leader Bạn cần hiểu biết về công nghệ sâu và rộng vì chính bạn là người lựa chọn công nghệ, quy trình... cho một dự án. Những quyết định lớn về thiết kế, cấu trúc code ... sẽ do bạn chịu trách nhiệm. Ở giai đoạn này, ngoài việc technical “cứng”, bạn còn phải giỏi thuyết trình, hướng dẫn, giải thích... Vì sao